

Project 1:Real-Time Powershell Monitoring

Introduction

Real-Time PowerShell Monitoring using Splunk Enterprise is a SOC-focused project that demonstrates how Windows PowerShell Operational logs can be collected, monitored, and analyzed in real time using Splunk Enterprise and Splunk Universal Forwarder. The project focuses on detecting suspicious PowerShell activities through SPL queries, dashboards, and real-time alerts, providing hands-on experience in SIEM monitoring, log analysis, and threat detection.

Lab Setup

To complete this project, the following environment was used:

- Windows 11
- Splunk Enterprise
- Splunk Universal Forwarder
- Windows PowerShell
- Local Host Environment

Pre-requisites

Before starting the project, ensure the following requirements are met:

- Splunk Enterprise Installed
- Splunk Universal Forwarder Installed
- Administrator Access

- PowerShell Operational Logging Enabled

Tools Installation

1. Install Splunk Enterprise on the Windows machine.
2. Install Splunk Universal Forwarder.
3. Configure the Universal Forwarder to communicate with Splunk Enterprise.
4. Verify that both Splunk Enterprise and the Universal Forwarder services are running successfully.

Exercises

Exercise 1: Installing and Configuring Splunk Enterprise

Objective

Install Splunk Enterprise and configure the initial setup for real-time log collection and security monitoring.

Step 1: Download Splunk Enterprise

Download the latest Splunk Enterprise installer from the official Splunk website.

Step 2: Install Splunk Enterprise

Run the installer as an administrator and complete the installation process.

Step 3: Create an Administrator Account

Launch Splunk Enterprise and create an administrator username and password.

Step 4: Access the Splunk Web Interface

Open the Splunk Web interface using the default URL.

<http://localhost:8000>

Step 5: Verify the Installation

Confirm that Splunk Enterprise is running successfully and the dashboard is accessible.

Expected Output

- Splunk Enterprise is installed successfully.
- Administrator account is created.
- Splunk Web interface is accessible.
- Splunk is ready to collect and analyze log data.

Exercise 2: Installing and Configuring Splunk Universal Forwarder

Objective

Install and configure Splunk Universal Forwarder to send Windows PowerShell logs to Splunk Enterprise.

Step 1: Verify the Forwarder Service

Check whether the Splunk Universal Forwarder service is running.

```
sc query SplunkForwarder
```

Step 2: Check Forwarder Status

Verify that the forwarder is active.

```
"C:\Program Files\SplunkUniversalForwarder\bin\splunk.exe"  
status
```

Step 3: Configure the Forward Server

Connect the Universal Forwarder to Splunk Enterprise.

```
"C:\Program Files\SplunkUniversalForwarder\bin\splunk.exe" add  
forward-server 127.0.0.1:9997
```

Step 4: Restart the Forwarder

Apply the configuration changes.

```
"C:\Program Files\SplunkUniversalForwarder\bin\splunk.exe"  
restart
```

Expected Output

- Splunk Universal Forwarder is running successfully.
- The forwarder is connected to Splunk Enterprise.
- Windows PowerShell logs are forwarded in real time.

Exercise 3: PowerShell Log Collection and Verification

Objective

Enable PowerShell Operational Logging and verify that PowerShell events are collected successfully in Splunk Enterprise.

Step 1: Enable PowerShell Operational Logging

Enable PowerShell Operational Logging to capture PowerShell activities.

```
wevtutil sl "Microsoft-Windows-PowerShell/Operational" /e:true
```

Step 2: Verify Log Status

Check whether the PowerShell Operational log is enabled.

```
Get-WinEvent -ListLog *PowerShell* | Select-Object LogName, IsEnabled
```

Step 3: Generate PowerShell Events

Run a few PowerShell commands to generate log events.

```
Get-Process
```

```
Get-Service
```

```
Get-Date
```

Step 4: Verify Events in Splunk

Search for the collected PowerShell logs.

```
index=main sourcetype="WinEventLog:Microsoft-Windows-PowerShell/Operational"
```

Step 5: Filter Event ID 4104

Display only PowerShell Script Block Logging events.

```
index=main sourcetype="WinEventLog:Microsoft-Windows-
PowerShell/Operational" EventCode=4104
```

Expected Output

- PowerShell Operational Logging is enabled.
- PowerShell events are successfully forwarded to Splunk.
- Event ID **4104** is visible for analysis.

Exercise 4: PowerShell Log Analysis and Threat Detection

Objective

Analyze PowerShell logs and identify suspicious activities using SPL queries.

Step 1: View All PowerShell Events

Displays all collected PowerShell Operational logs.

```
index=main sourcetype="WinEventLog:Microsoft-Windows-
PowerShell/Operational"
```

Step 2: Filter Event ID 4104

Filters PowerShell Script Block Logging events.

```
index=main sourcetype="WinEventLog:Microsoft-Windows-
PowerShell/Operational" EventCode=4104
```

Step 3: Detect Suspicious Commands

Searches for commonly abused PowerShell keywords.

```
index=main sourcetype="WinEventLog:Microsoft-Windows-
PowerShell/Operational" EventCode=4104
```

```
("EncodedCommand" OR "DownloadString" OR "Invoke-Expression")
```

Step 4: Generate a Test Event

Creates a test event to validate the detection rule.

```
Write-Output "EncodedCommand DownloadString Invoke-Expression
test"
```

Step 5: Verify Detection

Confirms that the generated event is detected in Splunk.

```
index=main sourcetype="WinEventLog:Microsoft-Windows-  
PowerShell/Operational" EventCode=4104
```

```
("EncodedCommand" OR "DownloadString" OR "Invoke-Expression")
```

Expected Output

- PowerShell events are successfully analyzed.
- Suspicious PowerShell activity is detected.
- Detection rule is validated successfully.

Exercise 5: Dashboard Creation and Real-Time Alert Configuration

Objective

Create a security dashboard and configure a real-time alert to monitor suspicious PowerShell activity.

Step 1: Create an Activity Timeline

Visualize PowerShell events over time.

```
index=main sourcetype="WinEventLog:Microsoft-Windows-  
PowerShell/Operational"
```

```
| timechart count
```

Step 2: View Event Distribution

Display the number of events by Event ID.

```
index=main sourcetype="WinEventLog:Microsoft-Windows-  
PowerShell/Operational"
```

```
| stats count by EventCode
```

Step 3: Create a Detection Alert

Use the detection query and save it as a **Real-Time Alert**.

```
index=main sourcetype="WinEventLog:Microsoft-Windows-  
PowerShell/Operational" EventCode=4104
```

```
("EncodedCommand" OR "DownloadString" OR "Invoke-Expression")
```

Alert Configuration

- **Alert Type:** Real-Time
- **Trigger:** Per Result
- **Alert Name:** Suspicious PowerShell Activity

Expected Output

- Dashboard displays real-time PowerShell activity.
- Suspicious PowerShell events are detected automatically.
- Real-time alert is triggered when matching activity is found.

splunk> Enterprise

New Search Save As Create Table View Close

Index=main sourcetype=WinEventLog:Microsoft-Windows-PowerShell/Operational

Time range: Last 24 hours

✓ 68 events (8/10/26 5:30:00.000 PM to 8/11/26 5:44:15.000 PM) No Event Sampling

Events (68) Patterns Statistics Visualization

Timeline format Zoom Out Zoom to Selection Deselect

Format Show: 20 Per Page View: List

< Hide Fields All Fields

SELECTED FIELDS

- # host 1
- # source 1
- # sourcetype 1

INTERESTING FIELDS

- # ComputerName 1
- # EventCode 6
- # EventType 2
- # index 1
- # Keywords 1
- # linecount 6
- # LogName 1
- # Message 36
- # OpCode 3

i	Time	Event
>	8/11/26 5:02:39.639 PM	08/11/2026 05:02:39.639 PM LogName=Microsoft-Windows-PowerShell/Operational EventCode=4104 EventType=3 ComputerName=DESKTOP-E9I1BMU Show all 19 lines host = DESKTOP-E9I1BMU source = WinEventLog:Microsoft-Windows-PowerShell/Operational sourcetype = WinEventLog:Microsoft-Windows-PowerShell/Operational
>	8/11/26 4:47:29.818 PM	08/11/2026 04:47:29.818 PM LogName=Microsoft-Windows-PowerShell/Operational EventCode=48962 EventType=4 ComputerName=DESKTOP-E9I1BMU Show all 15 lines host = DESKTOP-E9I1BMU source = WinEventLog:Microsoft-Windows-PowerShell/Operational

New Search Save As Create Table View Close

Index=main sourcetype=WinEventLog:Microsoft-Windows-PowerShell/Operational* EventCode=4104
("EncodedCommand" OR "DownloadString" OR "Invoke-Expression")

Time range: Last 24 hours

✓ 3 events (8/10/26 5:30:00.000 PM to 8/11/26 6:10:42.000 PM) No Event Sampling

Events (3) Patterns Statistics Visualization

Timeline format Zoom Out Zoom to Selection Deselect

Format Show: 20 Per Page View: List

< Hide Fields All Fields

SELECTED FIELDS

- # host 1
- # source 1
- # sourcetype 1

INTERESTING FIELDS

- # ComputerName 1
- # EventCode 1
- # EventType 1
- # index 1
- # Keywords 1
- # linecount 1
- # LogName 1
- # Message 3
- # OpCode 1
- # punct 1

i	Time	Event
>	8/11/26 6:10:30.711 PM	08/11/2026 06:10:30.711 PM ... 12 lines omitted ... OpCode=On create calls Message=Creating Scriptblock text (1 of 1): Write-Output "EncodedCommand test" Show all 19 lines host = DESKTOP-E9I1BMU source = WinEventLog:Microsoft-Windows-PowerShell/Operational sourcetype = WinEventLog:Microsoft-Windows-PowerShell/Operational
>	8/11/26 5:59:48.316 PM	08/11/2026 05:59:48.316 PM ... 12 lines omitted ... OpCode=On create calls Message=Creating Scriptblock text (1 of 1): Write-Output "EncodedCommand test" Show all 19 lines

```
PS C:\Windows\system32> wevtutil enum-logs | findstr /i powershell
Microsoft-Windows-PowerShell-DesiredStateConfiguration-FileDownloadManager/Analytic
Microsoft-Windows-PowerShell-DesiredStateConfiguration-FileDownloadManager/Debug
Microsoft-Windows-PowerShell-DesiredStateConfiguration-FileDownloadManager/Operational
Microsoft-Windows-PowerShell/Admin
Microsoft-Windows-PowerShell/Analytic
Microsoft-Windows-PowerShell/Debug
Microsoft-Windows-PowerShell/Operational
Windows PowerShell
PS C:\Windows\system32> Write-Output "EncodedCommand DownloadString Invoke-Expression test"
EncodedCommand DownloadString Invoke-Expression test
PS C:\Windows\system32>
```


